

扫描线

```
1 // Bentley-Ottmann: 判断多条线段是否存在任意交点,  $O(n \log n)$ 
2 bool segs_inter(const vector<Segment> &segs) {
3     if(segs.empty()) return false;
4     using seq_t=tuple<T,int,Segment>; // (x坐标, 左/右端点0/1, 线段)
5     const auto seqcmp = [](const seq_t &u, const seq_t &v) {
6         const auto [u0, u1, u2] = u;
7         const auto [v0, v1, v2] = v;
8         if(abs(u0 - v0) <= eps) return make_pair(u1, u2) < make_pair(v1, v2);
9         return u0 < v0 - eps;
10    };
11    vector<seq_t> seq;
12    for(auto seg: segs) {
13        if(seg.a.x > seg.b.x + eps) swap(seg.a, seg.b);
14        seq.push_back({seg.a.x, 0, seg});
15        seq.push_back({seg.b.x, 1, seg});
16    }
17    sort(seq.begin(), seq.end(), seqcmp);
18    T x_now; // 当前扫描线的 x 坐标
19    // 根据在当前扫描线处 y 的大小对线段进行排序
20    auto cmp = [&](const Segment &u, const Segment &v) {
21        if(abs(u.a.x - u.b.x) <= eps || abs(v.a.x - v.b.x) <= eps)
22            return u.a.y < v.a.y - eps;
23        return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) *
24            (v.b.x - v.a.x) < ((x_now - v.a.x) * (v.b.y - v.a.y) + v.a.y * (v.b.x -
25            v.a.x)) * (u.b.x - u.a.x) - eps;
26    };
27    multiset<Segment, decltype(cmp)> s{cmp};
28    for(const auto [x,o,seg]: seq) {
29        x_now = x;
30        const auto it = s.lower_bound(seg);
31        if(o == 0) {
32            if(it != s.end() && seg.is_inter(*it)) return true;
33            if(it != s.begin() && seg.is_inter(*prev(it))) return true;
34            s.insert(seg);
35        } else {
36            if(next(it) != s.end() && it != s.begin() &&
37            (*prev(it)).is_inter(*next(it))) return true;
38            s.erase(it);
39        }
40    }
41    return false;
42 }
```

```

41 #ifndef usedouble
42 // 多边形面积并:格林公式 (轮廓积分法),  $O(n^2 \log n)$ ,  $n$  为边数
43 // 返回值: ans[i] 表示平面内被至少覆盖了 i+1 次的区域的面积
44 vector<T> area_union(const vector<Polygon> &polys) {
45     const size_t siz = polys.size();
46     vector<vector<pair<Point, Point>>> segs(siz);
47     const auto check = [](const Point &u, const Segment &e) {
48         return !((u < e.a && u < e.b) || (u > e.a && u > e.b));
49     };
50     auto cut_edge = [&](const Segment &e, const size_t i) {
51         const Line le{e.a, e.b - e.a};
52         vector<pair<Point, int>> evt;
53         evt.push_back({e.a, 0}); evt.push_back({e.b, 0});
54         for(size_t j = 0; j < polys.size(); j++) {
55             if(i == j) continue;
56             const auto &pj = polys[j];
57             for(size_t k = 0; k < pj.p.size(); k++) {
58                 const Segment s = {pj.p[k], pj.p[pj.nxt(k)]};
59                 if(le.toleft(s.a) == 0 && le.toleft(s.b) == 0) {
60                     evt.push_back({s.a, 0});
61                     evt.push_back({s.b, 0});
62                 } else if (s.is_inter(le)) {
63                     const Line ls{s.a, s.b - s.a};
64                     const Point u = le.inter(ls);
65                     if(le.toleft(s.a) < 0 && le.toleft(s.b) >= 0)
66                         evt.push_back({u, -1});
67                     else if(le.toleft(s.a) >= 0 && le.toleft(s.b) < 0)
68                         evt.push_back({u, 1});
69                 }
70             }
71         }
72         sort(evt.begin(), evt.end());
73         if(e.a > e.b) reverse(evt.begin(), evt.end());
74         int sum = 0;
75         for(size_t i = 0; i < evt.size(); i++) {
76             sum += evt[i].second;
77             const Point u = evt[i].first, v = evt[i + 1].first;
78             if(!(u == v) && check(u, e) && check(v, e))
79                 segs[sum].push_back({u, v});
80             if(v == e.b) break;
81         }
82     };
83     for(size_t i = 0; i < polys.size(); i++) {
84         const auto &pi = polys[i];
85         for(size_t k = 0; k < pi.p.size(); k++) {
86             const Segment ei = {pi.p[k], pi.p[pi.nxt(k)]};
87             cut_edge(ei, i);
88         }
89     }
90 }

```

```
85     }
86 }
87 vector<T> ans(siz);
88 for(size_t i = 0; i < siz; i++) {
89     T sum = 0;
90     sort(segs[i].begin(), segs[i].end());
91     int cnt = 0;
92     for(size_t j = 0; j < segs[i].size(); j++) {
93         if(j > 0 && segs[i][j] == segs[i][j - 1])
94             segs[i + (++cnt)].push_back(segs[i][j]);
95         else cnt = 0, sum += segs[i][j].first ^ segs[i][j].second;
96     }
97     ans[i] = sum / 2;
98 }
99 return ans;
100 }
101 #endif
```